Python Programming

Intern

Name: Amima Batool

Starting Date: August 15, 2026

Ending Date: September 15, 2026

Project no. : Second

Project name: Expense Tracker Application

Expense Tracker Application

Introduction:

The Expense Tracker Application is a command-line application developed using Python. This application allows users to manage their daily expenses. Users can add expenses, view all expenses, update an expense, remove expense and calculate total expense. This project is developed as the part of the Python programming internship at DecodeLabs.

This projects helps to practice fundamental Python concepts, Math operations and build a practical beginner level application. This project provides practical hand-on experience in developing a Python application while applying concepts like lists, dictionary, arithmetic operations, functions, loops, conditional statements and error handling.

Tools & Technologies Used:

- VS Code
- Python
- GitHub

Objectives of the Project:

The main objectives of the Expense Tracker application are:

- To practice fundamental Python programming concepts.
- To learn how to store, update,delete and manage multiple tasks.
- To understand how functions are used to make program logic.
- To handle user input, output and error handling.
- To practice arithmetic logic.
- loops and conditional statements.
- To gain hand-on experience in building a project.
- To develop a command-line application.

Features:

Following are the features of the Expense Tracker Application:

1. Add an Expense:

Users can add a new expense by expense name with details like category, amount, date and description.

2. View All Expenses:

Users can view all stored expenses along with their details like category, amount, date and description.

3. Update an Expense:

Users can update their any expense with their details from the list.

4. View Total Spending:

Users can view their total spending.

5. Remove an Expense:

Users can remove an expense from the expense list.

6. Exit the Application:

Users can exit the application through the exit option given in menu.

Python Concepts Used:

Following fundamental Python Concepts are used in this project:

- **Lists:**

Lists are used to store and manage multiple expenses in the applications.

```
#======Expense Tracker======

#List to store All Expenses
expense_List = []
```

Here, ***append()*** is the list method to store the expenses with their details like category, amount, date and description in the list.

```
#Function to Add a New Expense
def add_expense():
    expense_name = input("Enter a new Expense Item: ")
    date = input("Enter Date (MM DD, YYYY): ")
    category = input("Enter Category: ")
    expense_amount = float(input("Enter amount: "))
    description = input("Enter description(optional): ")
    expense_List.append({"Expense": expense_name,
                         "Date": date,
                         "Category": category,
                         "Amount": expense_amount,
                         "Description": description})
    print()
    print("New Expense Added Successfully!!")
    print()
```

- **Dictionary:**

Dictionaries are used to organize the detail related to expenses. As, here we used dictionary to organize expenses and details.

```
#Function to Add a New Expense
def add_expense():
    expense_name = input("Enter a new Expense Item: ")
    date = input("Enter Date (MM DD, YYYY): ")
    category = input("Enter Category: ")
    expense_amount = float(input("Enter amount: "))
    description = input("Enter description(optional): ")
    expense_List.append({"Expense": expense_name,
                         "Date": date,
                         "Category": category,
                         "Amount": expense_amount,
                         "Description": description})
    print()
    print("New Expense Added Successfully!!")
    print()
```

➢ Functions:

Functions are used to divide the program into reusable sections. Different user defined functions are created to perform different operations like adding expenses, viewing all expenses, removing an expense,update an expense and view total spending.

```python
#-----FUNCTION DEFINATIONS-----

#Function to Add a New Expense
def add_expense():
    expense_name = input("Enter a new Expense Item: ")
    date = input("Enter Date (MM DD, YYYY): ")
    category = input("Enter Category: ")
    expense_amount = float(input("Enter amount: "))
    description = input("Enter description(optional): ")
    expense_List.append({"Expense": expense_name,
                         "Date": date,
                         "Category": category,
                         "Amount": expense_amount,
                         "Description": description})
    print()
    print("New Expense Added Successfully!!")
    print()


#Function to View All Expenses
def view_expenses():
    print("Your All Expenses: ")
    if len(expense_List) == 0:
        print("No Expenses Added yet!! ")
    else:
        for index, expense in enumerate(expense_List, 1):
            print("No. - NAME - AMOUNT - DATE - CATEGORY - DESCRIPTION ")
            print(f"{index}. {expense['Expense']} - {expense['Amount']} - {expense['Date']} - {expense['Category']} - {expense['Description']}")
            print()
    print()
```

```python
#Function to Update an Expense
def update_expense():
    index = int(input("Enter the expense number you want to update: ")) - 1

    if 0 <= index < len(expense_List):
        expense = expense_List[index]

        expense["Expense"] = input("Enter new expense name: ")
        expense["Date"] = input("Enter new date (MM DD, YYYY): ")
        expense["Category"] = input("Enter new category: ")
        expense["Amount"] = float(input("Enter new amount: "))
        expense["Description"] = input("Enter new description: ")
        print()
        print("Expense updated successfully!!")
        print()
    else:
        print("Invalid expense number!")
        print()


#Function to view Total Spending
def view_total_spending():
    total_spending = 0
    for expense in expense_List:
        total_spending = total_spending + expense['Amount']
    print(f"Total Spending:  {total_spending}")
    print()
    print("Expenses added successfully!!")
    print()
```

```python
#Function to Delete an Expense
def remove_expense():
    if len(expense_List) == 0:
        print("No Expenses to Remove!")
    else:
        index = int(input("Enter the expense number you want to remove: ")) - 1
        if 0 <= index < len(expense_List):
            removed_expense = expense_List.pop(index)
            print(f"Expense Removed: {removed_expense['Expense']}")
        else:
            print("Invalid expense number!")
    print()
```

➢ Loops:

Loops are used to repeatedly display the menu and allow the users to perform multiple operations without restarting the whole program.

```python
#Function to display a menu
def menu():
    while(True):
        print("----Main Menu----")
        print("1. Add a New Expense")
        print("2. View all Expenses")
        print("3. Update an Expense")
        print("4. View Total Spending")
        print("5. Remove an Expense")
        print("6. Exit")
        print()

        choice = input("Enter your choice: ")
        print()
        if choice == "1":
            add_expense()
        elif choice =="2":
            view_expenses()
        elif choice == "3":
            update_expense()
        elif choice =="4":
            view_total_spending()
        elif choice =="5":
            remove_expense()
        elif choice =="6":
            print("Exiting the application....")
            print()
            exit()
        else:
            print("Invalid choice! Try again!!")
            print()
```

- **Conditional Statements:**

Conditional statements are used to make decisions based on the user selection.

```python
    choice = input("Enter your choice: ")
    print()
    if choice == "1":
        add_expense()
    elif choice =="2":
        view_expenses()
    elif choice == "3":
        update_expense()
    elif choice =="4":
        view_total_spending()
    elif choice =="5":
        remove_expense()
    elif choice =="6":
        print("Exiting the application....")
        print()
        exit()
    else:
        print("Invalid choice! Try again!!")
        print()


nu()
```

```python
    if 0 <= index < len(expense_List):
        expense = expense_List[index]

        expense["Expense"] = input("Enter new expense name: ")
        expense["Date"] = input("Enter new date (MM DD, YYYY): ")
        expense["Category"] = input("Enter new category: ")
        expense["Amount"] = float(input("Enter new amount: "))
        expense["Description"] = input("Enter new description: ")
        print()
        print("Expense updated successfully!!")
        print()
    else:
        print("Invalid expense number!")
        print()
```

```python
def remove_expense():
    if len(expense_List) == 0:
        print("No Expenses to Remove!")
    else:
        index = int(input("Enter the expense number you want to remove: ")) - 1
        if 0 <= index < len(expense_List):
            removed_expense = expense_List.pop(index)
            print(f"Expense Removed: {removed_expense['Expense']}")
        else:
            print("Invalid expense number!")
    print()
```

Working of the Program:

The Expense Tracker Application works in a step-by-step manner to perform calculation operations. The execution process of the program is as follows:

1. **Display Menu:**

When the program starts, a menu is displayed in the terminal. The user selects an option according to its need.

```python
#Function to display a menu
def menu():
    while(True):
        print("----Main Menu----")
        print("1. Add a New Expense")
        print("2. View all Expenses")
        print("3. Update an Expense")
        print("4. View Total Spending")
        print("5. Remove an Expense")
        print("6. Exit")
        print()

        choice = input("Enter your choice: ")
        print()
        if choice == "1":
            add_expense()
        elif choice =="2":
            view_expenses()
        elif choice == "3":
            update_expense()
        elif choice =="4":
            view_total_spending()
        elif choice =="5":
            remove_expense()
        elif choice =="6":
            print("Exiting the application....")
            print()
            exit()
        else:
            print("Invalid choice! Try again!!")
            print()


menu()
```

A *menu()* function is created, which is called when program is run. Loop is used to print menu every time until the user choose to exit the program from the menu.

2. Select an Option:

When the menu displayed in the terminal, there are five different option for user to choose.

```
----Main Menu----
1. Add a New Expense
2. View all Expenses
3. Update an Expense
4. View Total Spending
5. Remove an Expense
6. Exit

Enter your choice:
```

- **Add a New Expense:**

User defined function ***add_expense()*** is created to add a new expense in the list. The list method ***append()*** is used to store all the expenses in the list. In the ***append(),*** the dictionary is used to store expense with details like category, amount, date and description.

```python
#Function to Add a New Expense
def add_expense():
    expense_name = input("Enter a new Expense Item: ")
    date = input("Enter Date (MM DD, YYYY): ")
    category = input("Enter Category: ")
    expense_amount = float(input("Enter amount: "))
    description = input("Enter description(optional): ")
    expense_List.append({"Expense": expense_name,
                         "Date": date,
                         "Category": category,
                         "Amount": expense_amount,
                         "Description": description})
    print()
    print("New Expense Added Successfully!!")
    print()
```

The user selects the option from the menu to add a new expense and enters the details. The expense will be store.

```
Enter your choice: 1

Enter a new Expense Item: Pencil
Enter Date (MM DD, YYYY): 08 26, 2026
Enter Category: Writing Utensil
Enter amount: 78
Enter description(optional): Bought from School Shop

New Expense Added Successfully!!
```

```
Enter your choice: 1

Enter a new Expense Item: Pizza
Enter Date (MM DD, YYYY): 08 27, 2026
Enter Category: Food
Enter amount: 842
Enter description(optional): Ordered Online

New Expense Added Successfully!!
```

- **View All Expenses:**

User defined function ***view_expenses()*** is created to view all the expenses stored in the list. In this function, if-condition is used to check whether the list is empty or not. If list is not empty, all expenses with their details in the list will view.

```
#Function to View All Expenses
def view_expenses():
    print("Your All Expenses: ")
    if len(expense_List) == 0:
        print("No Expenses Added yet!! ")
    else:
        for index, expense in enumerate(expense_List, 1):
            print("No. - NAME - AMOUNT - DATE - CATEGORY - DESCRIPTION ")
            print(f"{index}. {expense['Expense']} - {expense['Amount']} - {expense['Date']} - {expense['Category']} - {expense['Description']}")
            print()
    print()
```

The user selects the option from the menu to view all expenses. The expense list along their status will appear.

```
----Main Menu----
1. Add a New Expense
2. View all Expenses
3. Update an Expense
4. View Total Spending
5. Remove an Expense
6. Exit

Enter your choice: 2

Your All Expenses:
No. - NAME - AMOUNT - DATE - CATEGORY - DESCRIPTION
1. Pencil - 78.0 - 08 26, 2026 - Writing Utensil - Bought from School Shop

No. - NAME - AMOUNT - DATE - CATEGORY - DESCRIPTION
2. Pizza - 842.0 - 08 27, 2026 - Food - Ordered Online
```

- **Update an Expense:**

User defined function ***update_expenses()*** is created to update any expense stored in the list. In this function, if-condition is used to check that enter expense number to update is present or not. If that expense is listed, then user can update it.

```python
#Function to Update an Expense
def update_expense():
    index = int(input("Enter the expense number you want to update: ")) - 1

    if 0 <= index < len(expense_List):
        expense = expense_List[index]

        expense["Expense"] = input("Enter new expense name: ")
        expense["Date"] = input("Enter new date (MM DD, YYYY): ")
        expense["Category"] = input("Enter new category: ")
        expense["Amount"] = float(input("Enter new amount: "))
        expense["Description"] = input("Enter new description: ")
        print()
        print("Expense updated successfully!!")
        print()
    else:
        print("Invalid expense number!")
        print()
```

The user selects the option from the menu to update the expense. The expense will be updated along with their details.

```
----Main Menu----
1. Add a New Expense
2. View all Expenses
3. Update an Expense
4. View Total Spending
5. Remove an Expense
6. Exit

Enter your choice: 3

Enter the expense number you want to update: 2
Enter new expense name: Pizza and Burger with soda
Enter new date (MM DD, YYYY): 08 27, 2026
Enter new category: Food
Enter new amount: 1597
Enter new description: Ordered Online

Expense updated successfully!!
```

- **View Total Spending:**

User defined function ***view_total_spending()*** is created to calculate total spending of expenses listed.

```python
#Function to view Total Spending
def view_total_spending():
    total_spending = 0
    for expense in expense_List:
        total_spending = total_spending + expense['Amount']
    print(f"Total Spending:  {total_spending}")
    print()
    print("Expenses added successfully!!")
    print()
```

The user selects the option from the menu to view total spending. Then, their total calculated spending will appear.

```
----Main Menu----
1. Add a New Expense
2. View all Expenses
3. Update an Expense
4. View Total Spending
5. Remove an Expense
6. Exit

Enter your choice: 4

Total Spending:  1675.0

Expenses added successfully!!
```

- **Delete an expense:**

User defined function ***remove_expense()*** is created to remove a expense from the list. In this function, if-condition is used to check rather the expense list is empty expense is removed.

```
#Function to Delete an Expense
def remove_expense():
    if len(expense_List) == 0:
        print("No Expenses to Remove!")
    else:
        index = int(input("Enter the expense number you want to remove: ")) - 1
        if 0 <= index < len(expense_List):
            removed_expense = expense_List.pop(index)
            print(f"Expense Removed: {removed_expense['Expense']}")
        else:
            print("Invalid expense number!")
    print()
```

The user selects the option from the menu to remove an expense. The expense along its details will be removed.

```
----Main Menu----
1. Add a New Expense
2. View all Expenses
3. Update an Expense
4. View Total Spending
5. Remove an Expense
6. Exit

Enter your choice: 5

Enter the expense number you want to remove: 2
Expense Removed: Pizza and Burger with soda
```

3. Exit the Application:

To exit the application, the built-in ***exit()*** function is used.

```
        elif choice =="5":
            remove_expense()
        elif choice =="6":
            print("Exiting the application....")
            print()
            exit()
        else:
```

```
----Main Menu----
1. Add a New Expense
2. View all Expenses
3. Update an Expense
4. View Total Spending
5. Remove an Expense
6. Exit

Enter your choice: 6

Exiting the application....
```

Application Flow:

The overall basic workflow of the application is as follow:

Start → Display menu() function → User Selects an option → Add/View/Update/Remove/Calculate → Perform operation → Display menu again → Exit Application → End.

Code:

```
#======Expense Tracker======
#List to store All Expenses
expense_List = []
#-----FUNCTION DEFINATIONS-----
#Function to Add a New Expense
def add_expense():
    expense_name = input("Enter a new Expense Item: ")
    date = input("Enter Date (MM DD, YYYY): ")
    category = input("Enter Category: ")
    expense_amount = float(input("Enter amount: "))
    description = input("Enter description(optional): ")
    expense_List.append({"Expense": expense_name,
                         "Date": date,
                         "Category": category,
                         "Amount": expense_amount,
                         "Description": description})
    print()
    print("New Expense Added Successfully!!")
    print()
#Function to View All Expenses
def view_expenses():
    print("Your All Expenses: ")
    if len(expense_List) == 0:
        print("No Expenses Added yet!! ")
    else:
        for index, expense in enumerate(expense_List, 1):
            print("No. - NAME - AMOUNT - DATE - CATEGORY - DESCRIPTION ")
            print(f"{index}. {expense['Expense']} - {expense['Amount']} - {expense['Date']} -
{expense['Category']} - {expense['Description']}")
            print()
    print()
#Function to Update an Expense
```

```
def update_expense():
    index = int(input("Enter the expense number you want to update: ")) - 1
    if 0 <= index < len(expense_List):
        expense = expense_List[index]
        expense["Expense"] = input("Enter new expense name: ")
        expense["Date"] = input("Enter new date (MM DD, YYYY): ")
        expense["Category"] = input("Enter new category: ")
        expense["Amount"] = float(input("Enter new amount: "))
        expense["Description"] = input("Enter new description: ")
        print()
        print("Expense updated successfully!!")
        print()
    else:
        print("Invalid expense number!")
        print()
#Function to view Total Spending
def view_total_spending():
    total_spending = 0
    for expense in expense_List:
        total_spending = total_spending + expense['Amount']
    print(f"Total Spending:  {total_spending}")
    print()
    print("Expenses added successfully!!")
    print()
#Function to Delete an Expense
def remove_expense():
    if len(expense_List) == 0:
        print("No Expenses to Remove!")
    else:
        index = int(input("Enter the expense number you want to remove: ")) - 1
        if 0 <= index < len(expense_List):
            removed_expense = expense_List.pop(index)
            print(f"Expense Removed: {removed_expense['Expense']}")
        else:
```

```
        print("Invalid expense number!")
    print()
#Function to display a menu
def menu():
    while(True):
        print("----Main Menu----")
        print("1. Add a New Expense")
        print("2. View all Expenses")
        print("3. Update an Expense")
        print("4. View Total Spending")
        print("5. Remove an Expense")
        print("6. Exit")
        print()
        choice = input("Enter your choice: ")
        print()
        if choice == "1":
            add_expense()
        elif choice =="2":
            view_expenses()
        elif choice == "3":
            update_expense()
        elif choice =="4":
            view_total_spending()
        elif choice =="5":
            remove_expense()
        elif choice =="6":
            print("Exiting the application....")
            print()
            exit()
        else:
            print("Invalid choice! Try again!!")
            print()
menu()   #Function Call
```

Sample Output:

```
PS C:\Users\amima\OneDrive\Desktop\VS CODE folder> python Expense_Tracker.py
----Main Menu----
1. Add a New Expense
2. View all Expenses
3. Update an Expense
4. View Total Spending
5. Remove an Expense
6. Exit

Enter your choice: 1

Enter a new Expense Item: Pensil
Enter Date (MM DD, YYYY): 08 26. 2026
Enter Category: Writing Utensil
Enter amount: 78
Enter description(optional): Bought from School Shop

New Expense Added Successfully!!

----Main Menu----
1. Add a New Expense
2. View all Expenses
3. Update an Expense
4. View Total Spending
5. Remove an Expense
6. Exit

Enter your choice: 1

Enter a new Expense Item: Pizza
Enter Date (MM DD, YYYY): 08 27, 2026
Enter Category: Food
Enter amount: 842
Enter description(optional): Ordered

New Expense Added Successfully!!

----Main Menu----
1. Add a New Expense
```

```
New Expense Added Successfully!!

----Main Menu----
1. Add a New Expense
2. View all Expenses
3. Update an Expense
4. View Total Spending
5. Remove an Expense
6. Exit

Enter your choice: 2

Your All Expenses:
No. - NAME - AMOUNT - DATE - CATEGORY - DESCRIPTION
1. Pensil - 78.0 - 08 26. 2026 - Writing Utensil - Bought from School Shop

No. - NAME - AMOUNT - DATE - CATEGORY - DESCRIPTION
2. Pizza - 842.0 - 08 27, 2026 - Food - Ordered


----Main Menu----
1. Add a New Expense
2. View all Expenses
3. Update an Expense
4. View Total Spending
5. Remove an Expense
6. Exit

Enter your choice: 3

Enter the expense number you want to update: 2
Enter new expense name: Pizza and Burger with soda
Enter new date (MM DD, YYYY): 08 27, 2026
Enter new category: Food
Enter new amount: 1597
Enter new description: Ordered Online

Expense updated successfully!!
```

```
Expense updated successfully!!

----Main Menu----
1. Add a New Expense
2. View all Expenses
3. Update an Expense
4. View Total Spending
5. Remove an Expense
6. Exit

Enter your choice: 4

Total Spending:  1675.0

Expenses added successfully!!

----Main Menu----
1. Add a New Expense
2. View all Expenses
3. Update an Expense
4. View Total Spending
5. Remove an Expense
6. Exit

Enter your choice: 5

Enter the expense number you want to remove: 2
Expense Removed: Pizza and Burger with soda

----Main Menu----
1. Add a New Expense
2. View all Expenses
3. Update an Expense
4. View Total Spending
5. Remove an Expense
6. Exit
```

```
----Main Menu----
1. Add a New Expense
2. View all Expenses
3. Update an Expense
4. View Total Spending
5. Remove an Expense
6. Exit

Enter your choice: 2

Your All Expenses:
No. - NAME - AMOUNT - DATE - CATEGORY - DESCRIPTION
1. Pensil - 78.0 - 08 26. 2026 - Writing Utensil - Bought from School Shop


----Main Menu----
1. Add a New Expense
2. View all Expenses
3. Update an Expense
4. View Total Spending
5. Remove an Expense
6. Exit

Enter your choice: 5

Enter the expense number you want to remove: 2
Invalid expense number!

----Main Menu----
1. Add a New Expense
2. View all Expenses
3. Update an Expense
4. View Total Spending
5. Remove an Expense
6. Exit

Enter your choice: 2
```

```
5. Remove an Expense
6. Exit

Enter your choice: 2

Your All Expenses:
No. - NAME - AMOUNT - DATE - CATEGORY - DESCRIPTION
1. Pensil - 78.0 - 08 26. 2026 - Writing Utensil - Bought from School Shop


----Main Menu----
1. Add a New Expense
2. View all Expenses
3. Update an Expense
4. View Total Spending
5. Remove an Expense
6. Exit

Enter your choice: 4

Total Spending:  78.0

Expenses added successfully!!

----Main Menu----
1. Add a New Expense
2. View all Expenses
3. Update an Expense
4. View Total Spending
5. Remove an Expense
6. Exit

Enter your choice: 6

Exiting the application....

PS C:\Users\amima\OneDrive\Desktop\VS CODE folder>
```

Testing:

The application was tested by performing each operation:

Sr.	Task	Result	Status
1.	Add a new Expense	Expense was added successfully.	Passed
2.	View all Expenses	Expense were viewed successfully with details.	Passed
3.	Update an Expense	Expense was updated successfully.	Passed
4.	View Total Spending	Total Spending was calculated and view successfully.	Passed
5.	Remove an Expense	Expense was removed successfully.	Passed
6.	Exit Application	Terminate Program successfully.	Passed

Conclusion:

The Expense Tracker project successfully demonstrates how Python can be used to develop a simple and practical console-based application. It allow to add, view, update, and delete expenses.

Through this project, I practiced important Python concepts such as functions, lists, dictionaries, loops, conditional statements, arithmetic operations, and user input handling.

GitHub Repository:

AmimaBatool/Task-2 AmimaBatool: A simple Python console-based Expense Tracker to record, manage and track daily expenses. 💰📊

Project status: Completed.